

Esercitazione 5

Osservazioni sui Tempi degli Algoritmi: Debug vs Release

1. Tabelle dei Tempi di Esecuzione

I seguenti dati sono stati raccolti eseguendo il programma su vettori preallocati, calcolando la media su 100 esecuzioni per ogni dimensione.

Tabella 1: Compilazione in modalità Debug

Dim	Ins	Merge	Quick	Ibrido	std::sort
10	5.7406e-07	6.06762e-06	2.82529e-06	1.42614e-06	1.28213e-06
20	2.25723e-06	1.53766e-05	2.17622e-06	3.28534e-06	2.14522e-06
50	7.59478e-06	2.06661e-05	5.50756e-06	7.99482e-06	8.34386e-06
100	2.59207e-05	4.22893e-05	1.79228e-05	1.67977e-05	1.51436e-05
500	0.000455417	0.000230554	7.69949e-05	0.00010241	9.58648e-05
1000	0.00169822	0.000469815	0.000168103	0.000219684	0.000211314

Tabella 2: Compilazione in modalità Release

Dim	Ins	Merge	Quick	Ibrido	std::sort
10	1.3103e-07	1.65831e-06	2.3305e-07	3.9807e-07	1.2802e-07
20	3.0406e-07	5.29199e-06	8.0515e-07	3.00557e-06	5.0009e-07
50	1.90236e-06	6.84428e-06	1.98737e-06	2.55848e-06	1.09721e-06
100	4.15778e-06	1.53189e-05	4.99894e-06	6.49721e-06	3.99374e-06
500	5.84079e-05	5.51593e-05	2.31083e-05	2.35604e-05	1.72712e-05
1000	0.000182256	9.32875e-05	5.23898e-05	5.91761e-05	4.69828e-05

2. Confronto e Analisi dei Risultati

Dall'osservazione incrociata dei dati empirici raccolti nelle due modalità di compilazione emergono le seguenti considerazioni:

- **L'Impatto dell'Ottimizzazione (Debug vs Release):** La differenza più evidente risiede nell'ordine di grandezza dei tempi. La compilazione in **Release** applica pesanti ottimizzazioni (come l'inlining delle funzioni e l'ottimizzazione dei cicli). A dimensione $N=1000$, il Quicksort passa da circa 0.00016s (Debug) a 0.00005s (Release), risultando oltre 3 volte più veloce. Anche l'Insertion Sort beneficia enormemente dell'ottimizzazione, passando da 0.0016s a 0.00018s (quasi 10 volte più veloce).

- **Efficienza sui piccoli array :** Come richiesto, è possibile individuare una dimensione sotto la quale gli algoritmi quadratici risultano più veloci. In entrambe le modalità, per $N=10$ e $N=20$ l'Insertion Sort batte o eguaglia gli algoritmi logaritmici. In Release, a $N=20$ l'Insertion Sort impiega $3.04 \cdot 10^{-7}s$ contro gli $8.05 \cdot 10^{-7}s$ del Quicksort. Questo avviene perché la complessità $\mathcal{O}(n^2)$ su numeri piccolissimi maschera il suo peso, mentre l'overhead delle chiamate ricorsive (tipico di Merge e Quick) si fa sentire. La soglia ideale in cui avviene il "sorpasso" si attesta intorno ai 30-50 elementi.
- **Il Quicksort Ibrido:** Implementando un Quicksort Ibrido (che utilizza Insertion Sort per i sotto-array sotto una certa soglia) si cerca di unire i vantaggi dei due approcci. I dati mostrano che si difende molto bene, rimanendo competitivo in quasi tutti gli scenari, sebbene in Release le ottimizzazioni di base rendano già il Quicksort puro estremamente veloce.
- **Comportamento asintotico sui grandi array:** Osservando $N=500$ e $N=1000$ si palesa inequivocabilmente l'andamento teorico della complessità computazionale. L'Insertion Sort cresce in modo quadratico, diventando nettamente l'algoritmo più lento (in Release impiega $1.82 \cdot 10^{-4}s$ a $N=1000$). Il Quicksort e `std::sort` crescono invece secondo un fattore $\mathcal{O}(n \log n)$, risultando i dominatori incontrastati sulle medie/grandi dimensioni.
- **Mergesort vs Quicksort:** Il Mergesort risulta sistematicamente più lento del Quicksort in ogni scenario e dimensione. Questo scarto non è dovuto alla complessità asintotica (entrambi sono $\mathcal{O}(n \log n)$ nel caso medio), bensì alle continue allocazioni e deallocazioni di memoria per i vettori temporanei L e R necessarie nella procedura MERGE.